Lec 3

16Th July, 2026

Data Structures Notes — Arrays & Searching

Part 1 — Linear Arrays: Memory Representation

1.1 Core Idea

A **linear array** is stored in **contiguous (consecutive) memory locations**. Because every element occupies the same amount of space (word size), the address of *any* element can be computed directly from its index, without walking through the array. This is why array access is **O(1)**, unlike a linked list, where you must traverse node by node.

1.2 The Master Formula

$$Loc(Arr[K]) = Base(Arr) + w \times (K - LB)$$

Symbol	Meaning
$Loc(Arr[K])$	Address of the element at index K
$Base(Arr)$	Address of the **first** element (index = Lower Bound)
w	Word size — bytes occupied by **one** element
K	Index of the element whose address you want
LB	Lower Bound — smallest valid index
UB	Upper Bound — largest valid index

Why it works: $(K - LB)$ tells you how many elements come *before* $Arr[K]$. Multiplying by w converts that count into a number of bytes. Adding that byte-offset to $Base$ lands exactly on $Arr[K]$.

1.3 Worked Example — Finding an Address

Problem: $Arr[5 \ldots 40]$, $Base(Arr) = 1200$, $w = 4$ bytes (an `int` array). Find $Loc(Arr[20])$.

Step 1 — Identify the knowns

$$Base(Arr) = 1200, \quad w = 4, \quad LB = 5, \quad UB = 40, \quad K = 20$$

Step 2 — Compute the offset in elements

$$K - LB = 20 - 5 = 15$$

$Arr[20]$ is the 15th element after $Arr[5]$.

Step 3 — Convert to a byte-offset

$$w \times (K - LB) = 4 \times 15 = 60$$

Step 4 — Add the offset to the base

$$Loc(Arr[20]) = 1200 + 60 = 1260$$

Answer: $Loc(Arr[20]) = 1260$

1.4 Calculating the Size of an Array

$$Size = UB - LB + 1$$

Why the $+1$? LB and UB are both valid, *inclusive* indices, so the array holds the element at LB, the element at UB, and everything between them. $(UB - LB)$ alone only counts the gaps between indices, not the indices themselves, so you must add 1 to count both endpoints.

Example: $Arr[5 \ldots 40]$

$$Size = 40 - 5 + 1 = 36 \text{ elements}$$

1.5 Calculating Total Memory Required

$$Total\ Memory = Size \times w = (UB - LB + 1) \times w$$

Example: $Arr[5 \ldots 40]$, $w = 4$

$$Size = 36, \qquad Total\ Memory = 36 \times 4 = 144 \text{ bytes}$$

1.6 All Variants — Solving for Different Unknowns

The master formula can be rearranged to solve for whichever quantity is unknown.

Find the Address, given $K, Base, w, LB$

$$Loc(Arr[K]) = Base + w(K - LB)$$

Example: $Base = 1000, w = 2, LB = 1, K = 10 \Rightarrow Loc(Arr[10]) = 1000 + 2(10 - 1) = 1018$

Find the Index K, given the address

$$K = \frac{Loc(Arr[K]) - Base}{w} + LB$$

Example: $Base = 1000, w = 2, LB = 1$, address $= 1024$

$$K = \frac{1024 - 1000}{2} + 1 = 12 + 1 = 13$$

Find the Base Address, given $Loc(Arr[K]), K, w, LB$

$$Base = Loc(Arr[K]) - w(K - LB)$$

Example: $Loc(Arr[15]) = 1260, w = 4, LB = 5 \Rightarrow Base = 1260 - 4(10) = 1220$

Find Word Size w, given $Loc(Arr[K]), Base, K, LB$

$$w = \frac{Loc(Arr[K]) - Base}{K - LB}$$

Example: $Loc(Arr[10]) = 1040, Base = 1000, LB = 0 \Rightarrow w = \frac{40}{10} = 4$

Find Lower Bound LB, given $Loc(Arr[K]), Base, w, K$

$$LB = K - \frac{Loc(Arr[K]) - Base}{w}$$

Example: $Loc(Arr[20]) = 1260, Base = 1200, w = 4 \Rightarrow LB = 20 - 15 = 5$

Find Upper Bound, given $Size$ and LB

$$UB = Size + LB - 1$$

Example: $Size = 36, LB = 5 \Rightarrow UB = 40$

Find Lower Bound, given $Size$ and UB

$$LB = UB - Size + 1$$

Example: $UB = 40, Size = 36 \Rightarrow LB = 5$

Find Size, given $Total\ Memory$ and w

$$Size = \frac{Total\ Memory}{w}$$

Example: $Total\ Memory = 144, w = 4 \Rightarrow Size = 36$

Find Word Size, given $Total\ Memory$ and $Size$

$$w = \frac{Total\ Memory}{Size}$$

Example: $Total\ Memory = 144, Size = 36 \Rightarrow w = 4$

Find Address of the Last Element

$$Loc(Arr[UB]) = Base + w(UB - LB) = Base + Total\ Memory - w$$

Example: $Base = 1200, w = 4, Total\ Memory = 144 \Rightarrow Loc(Arr[UB]) = 1200 + 144 - 4 = 1340$

Find the Last Byte Address

$$Last\ Byte = Base + Total\ Memory - 1$$

Example: $Base = 1200, Total\ Memory = 144 \Rightarrow Last\ Byte = 1343$

1.7 Quick-Reference Formula Sheet

Goal	Formula
Address of $Arr[K]$	$Loc(Arr[K]) = Base + w(K - LB)$
Index from Address	$K = \frac{Loc(Arr[K]) - Base}{w} + LB$
Base Address	$Base = Loc(Arr[K]) - w(K - LB)$
Word Size	$w = \frac{Loc(Arr[K]) - Base}{K - LB}$
Lower Bound (from address data)	$LB = K - \frac{Loc(Arr[K]) - Base}{w}$
Size (number of elements)	$Size = UB - LB + 1$
Upper Bound (from size)	$UB = Size + LB - 1$
Lower Bound (from size)	$LB = UB - Size + 1$
Total Memory	$Total\ Memory = Size \times w = (UB - LB + 1) \times w$
Size (from total memory)	$Size = \frac{Total\ Memory}{w}$
Word Size (from total memory)	$w = \frac{Total\ Memory}{Size}$
Address of last element	$Loc(Arr[UB]) = Base + w(UB - LB)$
Last byte used	$Last\ Byte = Base + Total\ Memory - 1$

1.8 Practice Q&A Bank

Q1. $Arr[10 \ldots 100]$, $Base = 2000$, $w = 8$. Find $Loc(Arr[50])$.

$$Loc(Arr[50]) = 2000 + 8(50 - 10) = 2320$$

Q2. $Arr[1 \ldots 25]$, $w = 2$, $Base = 500$. Find total memory.

$$Size = 25, \qquad Total\ Memory = 25 \times 2 = 50 \text{ bytes}$$

Q3. $Base = 500$, $w = 2$, $LB = 1$. Element at address 538. Which index?

$$K = \frac{538 - 500}{2} + 1 = 20$$

Q4. $Arr[LB \dots 50]$ has 40 elements. Find LB.

$$LB = 50 - 40 + 1 = 11$$

Q5. $Loc(Arr[8]) = 1624$, $Base = 1600$, $LB = 0$. Find w.

$$w = \frac{1624 - 1600}{8} = 3 \text{ bytes}$$

Q6. $Arr[0 \dots 99]$, $w = 4$, $Base = 5000$. Find address of last element.

$$Loc(Arr[99]) = 5000 + 4(99) = 5396$$

Q7. Total memory $= 200$ bytes, $w = 4$. Find element count.

$$Size = \frac{200}{4} = 50$$

Q8. $Arr[5 \dots 54]$, $Base = 3000$, $w = 4$. Find the last byte address.

$$Size = 50, \qquad Total\ Memory = 200, \qquad Last\ Byte = 3000 + 200 - 1 = 3199$$

Q9. $Loc(Arr[K]) = 1260$, $w = 4$, $K = 15$, $LB = 5$. Find $Base$.

$$Base = 1260 - 4(10) = 1220$$

Q10. Array occupies addresses 1000 to 1099 inclusive, $w = 4$. Find element count.

$$Total\ Memory = 100, \qquad Size = \frac{100}{4} = 25$$

1.9 Key Takeaways

- Arrays get **O(1) random access** because addresses are computed arithmetically, not by traversal.
- $(K - LB)$ always means "how many elements away from the first element" — the conceptual heart of every variant of the formula.
- $Size = UB - LB + 1$ — never forget the $+1$; it is the most common source of off-by-one errors.
- Every unknown ($Base, w, K, LB, UB, Loc, Size, Total\ Memory$) can be isolated algebraically, just like solving for a variable in an equation.

Part 2 — Linear Search

2.1 Definition

Linear Search (also called **Sequential Search**) is the simplest searching technique. It checks each element of the array **one by one, in order**, comparing it against the search key x, until either a match is found or the entire array has been examined.

It makes **no assumption** about the ordering of the data — it works on sorted or unsorted arrays alike. This generality is its main advantage, and also the reason it is slow: in the worst case, every single element must be inspected.

- **Best case:** $O(1)$ — the target is the first element.
- **Worst case:** $O(n)$ — the target is the last element, or absent entirely.
- **Average case:** $O(n)$

2.2 Algorithm

Given an array A with n elements $(A[1], A[2], \ldots, A[n])$ and a search key x:

```
LINEAR_SEARCH(A, n, x)

Step 1:  Set i = 1
Step 2:  If i > n, go to Step 7
Step 3:  If A[i] = x, go to Step 6
Step 4:  Set i = i + 1
Step 5:  Go to Step 2
Step 6:  Print "element x found at index i", go to Step 8
Step 7:  Print "element not found"
Step 8:  Exit
```

Reading the logic: Steps 2–5 form a loop. Step 2 is the loop's exit condition (index ran off the end). Step 3 is the loop's success condition (match found). Step 4 advances the index, and Step 5 sends control back to re-check the exit condition. This is a classic "check, compare, advance, repeat" structure.

2.3 Code

C

```c
#include <stdio.h>

int linearSearch(int arr[], int n, int x) {
```

```c
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) {
            return i;   // return index where found
        }
    }
    return -1;          // not found
}

int main() {
    int arr[] = {10, 25, 3, 47, 8, 19};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 47;

    int result = linearSearch(arr, n, x);

    if (result != -1)
        printf("Element %d found at index %d\n", x, result);
    else
        printf("Element %d not found\n", x);

    return 0;
}
```

C++

```cpp
#include <iostream>
using namespace std;

int linearSearch(int arr[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) {
            return i;
        }
    }
    return -1;
}

int main() {
    int arr[] = {10, 25, 3, 47, 8, 19};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 47;

    int result = linearSearch(arr, n, x);

    if (result != -1)
```

```
        cout << "Element " << x << " found at index " << result << endl;
    else
        cout << "Element " << x << " not found" << endl;

    return 0;
}
```

Python

```
def linear_search(arr, x):
    for i in range(len(arr)):
        if arr[i] == x:
            return i
    return -1

arr = [10, 25, 3, 47, 8, 19]
x = 47

result = linear_search(arr, x)

if result != -1:
    print(f"Element {x} found at index {result}")
else:
    print(f"Element {x} not found")
```

2.4 Code Explained in Depth

Function signature — `linearSearch(arr, n, x)` / `linear_search(arr, x)` The function takes the array, its length (`n` — not needed in Python since `len(arr)` gives it directly), and the value being searched for, `x`. It will return the **index** of `x` if found, or `-1` as a sentinel value meaning "not found." Using `-1` works because valid array indices are never negative, so it can never be confused with a real index.

The loop — `for (int i = 0; i < n; i++)` / `for i in range(len(arr))` This loop is the direct translation of Steps 1–5 of the algorithm, but written using 0-based indexing (as C, C++, and Python all use), instead of the 1-based indexing used in the pseudocode.

- `i = 0` is the initialization — equivalent to algorithm Step 1 (`Set i = 1`), just shifted down by one because array indices start at 0 in these languages.
- `i < n` is the loop condition — equivalent to the *negation* of Step 2 (`if i > n go to step 7`). The loop keeps running as long as `i` is still a valid index.
- `i++` is the increment — equivalent to Step 4 (`Set i = i+1`), executed automatically at the end of each iteration.

The comparison — `if (arr[i] == x)` / `if arr[i] == x` This is Step 3 of the algorithm. On every iteration, the current element `arr[i]` is compared against the target `x`. Because `==` is a single equality check, this line is doing exactly one unit of "work" per loop iteration — this is why the algorithm's time complexity is directly proportional to how many iterations run.

On match — `return i` Equivalent to Step 6 (`print... found at index i`). Returning immediately is important: it means the function does **not** keep scanning after finding a match — it stops as soon as possible, which is what gives linear search its best-case $O(1)$ behavior when the match is near the front.

After the loop — `return -1` This line only executes if the loop completes *without* ever hitting the `return i` line — i.e., every element was checked and none matched. This corresponds to Step 7 (`print element not found`). Structurally, placing this line *after* the loop body ensures it only runs once the loop has naturally exhausted every index — there's no need for an explicit "if i > n" check like Step 2, because that check is baked into the loop's own exit condition.

Why this design works: notice the algorithm never looks at more than one element per iteration and never "jumps around" — it purely walks forward. That sequential, no-assumptions nature is both the strength (works on unsorted data) and the weakness (can't skip ahead, unlike binary search) of this technique.

Part 3 — Binary Search

3.1 Definition

Binary Search is a much faster searching technique, but it comes with one strict precondition: **the array must already be sorted**. Instead of checking every element, binary search repeatedly divides the search range in half:

1. Look at the **middle** element of the current range.
2. If it equals the target, you're done.
3. If the target is **smaller**, the target (if present) must lie in the **left half** — so discard the right half entirely.
4. If the target is **larger**, discard the left half and continue in the **right half**.
5. Repeat on the remaining half until the target is found, or the range becomes empty.

Because the search space is cut in half on every step, binary search is dramatically faster than linear search for large arrays.

- **Best case:** $O(1)$ — the target is the first middle element checked.
- **Worst case:** $O(\log_2 n)$ — the range keeps halving until only one element remains.

- **Average case:** $O(\log_2 n)$
- **Precondition:** the array **must be sorted** (ascending, typically).

3.2 Algorithm

Given a **sorted** array $A[LB \dots UB]$ and a search key x:

```
BINARY_SEARCH(A, LB, UB, x)

Step 1:  Set beg = LB, end = UB, mid = (beg + end) / 2
Step 2:  Repeat Steps 3 and 4 while beg <= end AND A[mid] != x
Step 3:      If x < A[mid], set end = mid - 1
             Else, set beg = mid + 1
Step 4:      Set mid = (beg + end) / 2
Step 5:  If A[mid] = x, print "element x found at index mid"
         Else, print "element not found"
Step 6:  Exit
```

Reading the logic: `beg` and `end` mark the current boundaries of the search range; `mid` is always recalculated as the midpoint of that range. Step 3 is the decision that shrinks the range: comparing `x` to the middle element tells you which half to keep. The loop (Step 2) continues only while there's still a valid range left (`beg <= end`) and the target hasn't been found yet. Once the loop ends, Step 5 checks whether it ended because of a match or because the range became empty.

3.3 Code

C

```c
#include <stdio.h>

int binarySearch(int arr[], int n, int x) {
    int beg = 0, end = n - 1;

    while (beg <= end) {
        int mid = beg + (end - beg) / 2;   // avoids overflow vs (beg+end)/2

        if (arr[mid] == x) {
            return mid;
        } else if (x < arr[mid]) {
            end = mid - 1;
        } else {
            beg = mid + 1;
        }
```

```c
    }
    return -1;   // not found
}

int main() {
    int arr[] = {3, 8, 10, 19, 25, 47};  // must be sorted
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 25;

    int result = binarySearch(arr, n, x);

    if (result != -1)
        printf("Element %d found at index %d\n", x, result);
    else
        printf("Element %d not found\n", x);

    return 0;
}
```

C++

```cpp
#include <iostream>
using namespace std;

int binarySearch(int arr[], int n, int x) {
    int beg = 0, end = n - 1;

    while (beg <= end) {
        int mid = beg + (end - beg) / 2;

        if (arr[mid] == x) {
            return mid;
        } else if (x < arr[mid]) {
            end = mid - 1;
        } else {
            beg = mid + 1;
        }
    }
    return -1;
}

int main() {
    int arr[] = {3, 8, 10, 19, 25, 47};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 25;
```

```
    int result = binarySearch(arr, n, x);

    if (result != -1)
        cout << "Element " << x << " found at index " << result << endl;
    else
        cout << "Element " << x << " not found" << endl;

    return 0;
}
```

Python

```
def binary_search(arr, x):
    beg, end = 0, len(arr) - 1

    while beg <= end:
        mid = beg + (end - beg) // 2

        if arr[mid] == x:
            return mid
        elif x < arr[mid]:
            end = mid - 1
        else:
            beg = mid + 1

    return -1

arr = [3, 8, 10, 19, 25, 47]   # must be sorted
x = 25

result = binary_search(arr, x)

if result != -1:
    print(f"Element {x} found at index {result}")
else:
    print(f"Element {x} not found")
```

3.4 Code Explained in Depth

Setting up the boundaries — `beg = 0, end = n - 1` This is Step 1 of the algorithm. `beg` and `end` represent the current lower and upper indices of the range still being searched. Initially, this range is the entire array — index `0` to `n-1`. As the search proceeds, these two variables will move toward each other, shrinking the range.

The loop condition — `while (beg <= end)` This corresponds to the "`beg <= end`" part of Step 2. It is the algorithm's way of asking "is there still a valid range to search?" If `beg` ever exceeds `end`, it means the range has become empty — there is nothing left to check, and the target isn't in the array. Note that the algorithm's other loop condition, `A[mid] != x`, is handled differently in the code: instead of checking it in the `while`, the code checks for a match *inside* the loop and returns immediately — which is functionally equivalent but avoids evaluating `arr[mid]` before `mid` is computed on the first pass.

Computing the midpoint — `mid = beg + (end - beg) / 2` This is Step 1's initialization and Step 4's recalculation combined into one line, executed at the top of every loop iteration. Mathematically this is identical to `(beg + end) / 2`, but it's written as `beg + (end - beg) / 2` to avoid **integer overflow** in languages like C/C++ when `beg` and `end` are both very large — adding them directly could overflow the integer type before the division happens. Because integer division truncates, `mid` always rounds down to the nearest whole index.

The three-way comparison — `if (arr[mid] == x) ... else if (x < arr[mid]) ... else ...` This block is the heart of Step 3 and Step 5 combined:

- `arr[mid] == x` — the target has been found. Return `mid` immediately (this is Step 5's success branch, but checked eagerly every iteration rather than waiting for the loop to end).
- `x < arr[mid]` — the target, if it exists, must be smaller than the middle element, so it can only be in the **left half**. The code discards the right half by moving `end` to `mid - 1` (excluding `mid` itself, since it's already been checked and ruled out).
- The final `else` (implicitly, `x > arr[mid]`) — the target must be in the **right half**, so `beg` moves to `mid + 1`.

This comparison is exactly why the array **must be sorted**: the logic "target is smaller, so it must be on the left" is only valid if everything to the left of `mid` is guaranteed to be smaller than `arr[mid]`, and everything to the right is guaranteed to be larger. On an unsorted array, discarding half the array like this would be invalid — the target could be anywhere.

Why the range shrinks so fast: each iteration eliminates roughly *half* of whatever range was still being searched, not just one element like linear search does. This is what produces the $O(\log_2 n)$ complexity — the number of times you can halve n before reaching a single element is $\log_2 n$.

After the loop — `return -1` Just like in linear search, this line only runs if the loop exits *without* returning — meaning `beg` crossed past `end` and the range became empty without ever matching `x`. This corresponds to the "not found" branch of Step 5.